



Security Review Report for Automated Liquidity Manager

August 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
5. Findings Summary
6. Weaknesses
 - Active ID slippage can omit liquidity
 - Deposits can force liquidity into idle
 - Shifted compact bin IDs cause incorrect position-limit quotes

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for SquadSwap protocol, a fork of PancakeSwap Infinity that adds more functionality, such as the Automated Liquidity Manager (ALM).

This report includes the findings on the ALM.

Our security assessment was a full review of the code, spanning a total of 1.5 weeks.

During our review, we did not identify any major severity vulnerability.

We did identify some minor severity vulnerabilities and code optimizations.

All reported issues were fixed by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

- **Review Led by**

Kasper Zwijsen, Head of Audits

- **Scope**

The analyzed resources are located on:

<https://github.com/skyrocktech/PositionManagerV4Contracts/tree/d869134dc84c6def6061207197a63a20f28e9333>

The issues described in this report were fixed in the following commit:

<https://github.com/skyrocktech/PositionManagerV4Contracts/tree/11ee58c956a36c20feaf11408ca9f6f74ab66db5>

- **Changelog**

■ 17 August 2026	Audit start
■ 1 September 2026	Initial report
■ 18 September 2026	Revision received
■ 22 September 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity

Number of findings

Critical	0
High	0
Medium	1
Low	2
Informational	0

Total:

3



Medium

Low



Fixed

6. Weaknesses

This section contains the list of discovered weaknesses.

SQUAD1-12 | Active ID slippage can omit liquidity

Fixed 

Severity:

Medium

Probability:

Rare

Impact:

High

Path:

`contracts/Strategy.sol:enterPosition#L1093-L1312`

Description:

The operator uses **enterPosition** to swap the strategy's assets if needed and add them as liquidity to SquadSwap bins. The function then records the minted ERC-1155 bin IDs and balances so that AUM calculations, user withdrawals, and operator exits can find the position later.

The recorded IDs should match the bins where liquidity was actually minted. This makes sure every asset removed from **reserveX** and **reserveY** remains included in **getAUMWithFees()** and can be recovered through the normal exit flow.

However, the strategy assumes every minted ID is **activeIdDesired + deltaIds[i]** and only checks balances at those IDs. SquadSwap instead mints at **currentActiveId + deltaIds[i]**; **activeIdDesired** is only used to check whether the current ID is within **idSlippage**. If the active ID moves inside that allowed range, the position is minted into different bins than the strategy records.

The assets used by SquadSwap are still removed from the strategy reserves, while the actual ERC-1155 balances are left out of **tokenBalances**. With zero or missing per-bin minimums, the call does not revert. The omitted liquidity is then excluded from AUM and cannot be burned by withdrawals, **exitPosition**, or **partialExitPosition**. A partial mismatch can also understate the share price and give later depositors too many shares.

For example:

1. The operator enters a one-bin position with **activeIdDesired = D**, **deltaIds = [0]**, **idSlippage = 1**, and a zero minimum.
2. A public trade or the strategy's own swap moves the pool's active ID to **D + 1** before liquidity is added.
3. SquadSwap accepts the move and mints the strategy's position at **D + 1**.

4. The strategy checks only bin **D**, records a zero balance, and still subtracts the deployed assets from its reserves.
5. AUM now omits the position, and **exitPosition** reverts with **NoPosition**. The liquidity stays stranded until the operator deliberately rediscovers the real bin in a later entry.

```
// Full source code excerpt omitted from the public version to protect proprietary implementation details
```

Remediation:

Record and validate the exact bin IDs minted by the position manager, and require **binIds**, **tokenBalances**, and **minLiquidities** to have matching lengths. If the integration cannot return the minted IDs, require **idSlippage** to be zero so the expected and actual IDs cannot differ.

SQUAD1-13 | Deposits can force liquidity into idle

Fixed ✓

Severity:

Low

Probability:

Unlikely

Impact:

Low

Path:

contracts/Strategy.sol:withdrawWithFees#L442-L621

Description:

The **withdrawWithFees** function lets shareholders redeem their shares for a proportional part of the strategy's reserves and LP position. Public strategy users can reach it after **_depositInternal** prices their deposit against the full AUM and leaves the deposited tokens in **reserveX** and **reserveY**.

When the reserves cannot fully cover a redemption, the function should remove only enough liquidity to cover the actual shortfall. A small change in a tracked bin should therefore not materially change the operator's LP allocation.

However, the function uses an all-or-nothing reserve check. If either reserve is short by even a negligible amount, it switches to the liquidity branch and burns the redeemer's full pro-rata share of every tracked bin:

// Full source code excerpt omitted from the public version to protect proprietary implementation details

An attacker can use this to replace a large part of the managed LP position with idle reserves:

1. The attacker deposits both assets in the strategy's AUM ratio and receives a large share of the supply. The assets stay in the reserves.
2. The attacker makes a small swap or one-sided donation through a tracked bin, increasing the strategy's claim to one token.
3. The attacker immediately calls **withdrawWithFees**. The changed claim makes one reserve slightly too small, so the function burns the attacker's full ownership fraction from every bin.
4. The attacker receives their fair pro-rata assets, but the same fraction of the previous LP position is left as idle reserves for the remaining holders. Any residual bin balance also prevents **enterPosition** from redeploying those reserves until the operator fully exits the position.

// Full source code excerpt omitted from the public version to protect proprietary implementation details

Remediation:

Remove only enough liquidity to cover the reserve shortfall on each token instead of burning the redeemer's full pro-rata share of every bin. Also clear zero or dust bin entries so the operator can redeploy the remaining reserves directly.

SQUAD1-14 | Shifted compact bin IDs cause incorrect position-limit quotes

Fixed ✓

Severity:

Low

Probability:

Unlikely

Impact:

Low

Path:

contracts/StrategyLens.sol:quoteCounterTokenAmount#L129-L183

Description:

The operator calls **quoteCounterTokenAmount** to work out how much of the counter token should be supplied when a strategy has a position limit. The function loads the strategy's canonical value distribution from **StrategyManager**, matches it to the submitted execution bins, and calculates the token ratio. The resulting quote is then passed to **Strategy.enterPosition** as **counterAmountMax**.

When zero-weight bins are removed, each remaining execution bin should keep the weight of its original canonical bin. This must also hold when the target price shifts the execution bin IDs relative to the pool's current active ID.

However, **_getCompactedValueDistribution** treats each shifted execution ID as if it were still a canonical ID:

```
// Full source code excerpt omitted from the public version to protect proprietary implementation details
```

This selects the wrong weights after a price shift and may also map past the end of the array, causing a revert. For example, **[0, 25, 25, 25, 25, 0]** shifted by one bin compacts to execution IDs **[-1, 0, 1, 2]**, all with weight **25**. The lens instead loads **[25, 25, 25, 0]**. With token0 as the primary token, this quotes a counter-token ratio of **1.00** instead of **0.60**, a 66.7% overquote.

An example flow is:

1. A strategy uses a position limit and a distribution containing zero-weight bins.
2. The target price differs from the pool's active ID, so the execution IDs are shifted, and the zero-weight bins are removed.
3. The operator calls **quoteCounterTokenAmount** with the compact execution IDs.
4. The lens loads weights using the shifted IDs and returns an incorrect quote or reverts.
5. If the operator uses the quote, **enterPosition** either leaves usable funds idle or supplies more counter tokens than the configured shape calls for. A revert blocks the normal re-entry or rebalance flow.

```
// Full source code excerpt omitted from the public version to protect proprietary implementation details
```

Remediation:

Keep each bin's canonical index when compacting and pass those indices, or the already-compacted weights, to the lens. Do not derive canonical distribution indices from shifted execution IDs.

hexens x  ALM

